Green University of Bangladesh
Department of Computer Science and Engineering (CSE)

Faculty of Sciences and Engineering
Semester: (Spring, Year: 2026), B.Sc. in CSE (Day)

Lab Report NO: 03
Course Title: AI Lab
Course Code: CSE 316
Section: 232-D5

Lab Experiment Name: Implementation of Min-Max and Alpha-Beta Algorithm

Student Details

	Name	ID
1.	Ashab Uddin	232002274

Lab Date : **08/04/2026**
Submission Date : **12/04/2026**
Course Teacher's Name : **Babe Sultana**

Lab Report Status

Marks: …… **Signature:.....**
Comments:..... **Date:.....**

1. INTRODUCTION

This lab report focuses on implementing adversarial search algorithms in Python, mainly the Minimax algorithm and Alpha-Beta Pruning. The main idea is to study how artificial intelligence makes decisions in a competitive environment where two players have opposite goals.

In this experiment, a complete binary game tree is constructed using a list of leaf node values provided by the user. The Minimax algorithm evaluates all possible outcomes of the tree to determine the best possible move. After that, Alpha-Beta Pruning is applied to optimize the search process by skipping branches that do not influence the final decision. The performance of both methods is compared using the number of nodes visited and pruned.

2. OBJECTIVES/AIM

- To understand the concept of Alpha-Beta in AI
- To implement Alpha-Beta Pruning in a Min Max search tree to optimize decision making in two player games.

3. PROCEDURE

• The program starts by taking input from the user in the form of leaf node values.
• These values represent final game outcomes in a binary tree structure.
• The system checks whether the number of inputs is a power of 2 to ensure a complete tree.
• The height of the tree is calculated using a logarithmic formula.

• First, the Minimax algorithm is executed.
• It recursively explores all nodes in the tree from root to leaf.
• At each level, MAX nodes choose the highest value while MIN nodes choose the lowest value.
• Every visited node is counted to measure total exploration cost.
• The final Minimax result represents the optimal decision.

• Next, Alpha-Beta Pruning is executed.
• It uses two values: alpha (best option for MAX) and beta (best option for MIN).
• If a branch cannot improve the result, it is ignored (pruned).
• This reduces unnecessary computation while maintaining the same final answer.
• Both visited and pruned nodes are recorded during execution.

• Finally, both algorithms are compared based on node exploration and efficiency improvement.

4. IMPLEMENTATION:

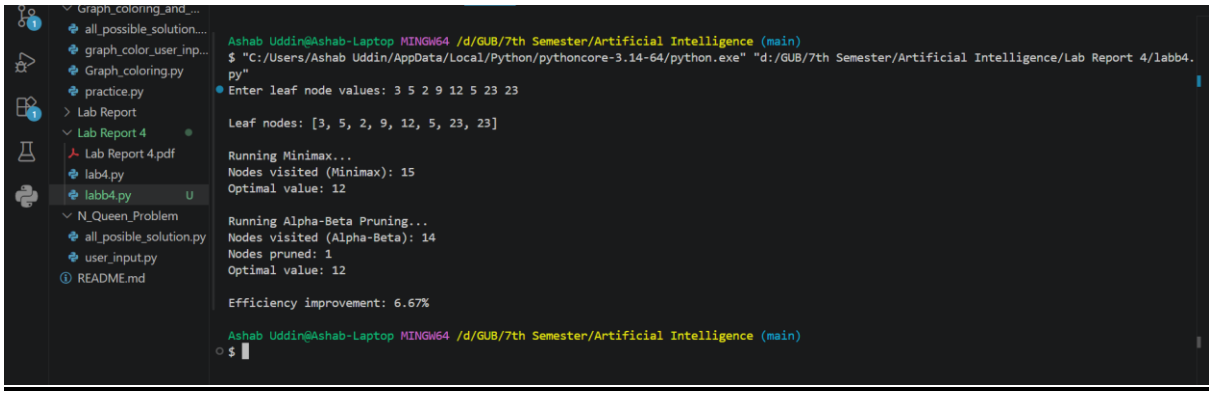
Code:

```
import math

# ---------------- MINIMAX ----------------
def minimax_search(level, index, max_turn, values, depth, stats):
    stats["visited"] += 1

    if level == depth:
        return values[index]

    if max_turn:
        left = minimax_search(level + 1, index * 2, False, values, depth,
stats)
        right = minimax_search(level + 1, index * 2 + 1, False, values, depth,
stats)
        return max(left, right)
    else:
        left = minimax_search(level + 1, index * 2, True, values, depth,
stats)
        right = minimax_search(level + 1, index * 2 + 1, True, values, depth,
stats)
        return min(left, right)


# ---------------- ALPHA-BETA ----------------
def alpha_beta_search(level, index, max_turn, values, depth, alpha, beta,
stats):
    stats["visited"] += 1

    if level == depth:
        return values[index]

    if max_turn:
        best = -float("inf")

        best = max(best, alpha_beta_search(level + 1, index * 2, False,
                                           values, depth, alpha, beta, stats))
        alpha = max(alpha, best)

        if alpha >= beta:
            stats["pruned"] += 2 ** (depth - level - 1)
            return best

        best = max(best, alpha_beta_search(level + 1, index * 2 + 1, False,
                                           values, depth, alpha, beta, stats))
        return best
```

```python
    else:
        best = float("inf")

        best = min(best, alpha_beta_search(level + 1, index * 2, True,
                                           values, depth, alpha, beta, stats))
        beta = min(beta, best)

        if alpha >= beta:
            stats["pruned"] += 2 ** (depth - level - 1)
            return best

        best = min(best, alpha_beta_search(level + 1, index * 2 + 1, True,
                                           values, depth, alpha, beta, stats))
        return best


# ---------------- MAIN PROGRAM ----------------
def run_game_tree():
    values = list(map(int, input("Enter leaf node values: ").split()))
    n = len(values)

    if n & (n - 1) != 0:
        print("Input size must be a power of 2!")
        return

    depth = int(math.log2(n))

    print("\nLeaf nodes:", values)

    # Minimax run
    print("\nRunning Minimax...")
    mini_stats = {"visited": 0}
    mini_result = minimax_search(0, 0, True, values, depth, mini_stats)

    print("Nodes visited (Minimax):", mini_stats["visited"])
    print("Optimal value:", mini_result)

    # Alpha-Beta run
    print("\nRunning Alpha-Beta Pruning...")
    ab_stats = {"visited": 0, "pruned": 0}
    ab_result = alpha_beta_search(0, 0, True, values, depth,
                                  -float("inf"), float("inf"), ab_stats)

    print("Nodes visited (Alpha-Beta):", ab_stats["visited"])
    print("Nodes pruned:", ab_stats["pruned"])
    print("Optimal value:", ab_result)

    # Efficiency
```

```
    improvement = ((mini_stats["visited"] - ab_stats["visited"])
                   / mini_stats["visited"]) * 100

    print("\nEfficiency improvement: {:.2f}%".format(improvement))


if __name__ == "__main__":
    run_game_tree()
```

5. OUTPUTS:



6. ANALYSIS AND DISCUSSION:

In this lab, we observed how Minimax explores every possible node in the game tree to determine the optimal solution. While it guarantees correctness, it is not efficient for large trees due to high computation cost.

Alpha-Beta Pruning improves this by eliminating branches that cannot influence the final decision. As a result, fewer nodes are visited while producing the same optimal output as Minimax.

The experiment clearly shows that Alpha-Beta Pruning is an optimized version of Minimax, reducing unnecessary exploration without affecting accuracy. The efficiency improvement percentage demonstrates how much computation was saved.

Overall, this lab helped strengthen understanding of adversarial search, recursive decision trees, and optimization techniques used in artificial intelligence game-playing systems.